DiveBuddy: A Local RAG Assistant Built with Microsoft Foundry Local

Final Report - Microsoft Summer Project | Summer 2026
Student: Tuna Kemer

---

1. Motivation

This project was undertaken to gain hands-on exposure to how modern AI applications are engineered, rather than relying on AI tools as a black box. Microsoft's Foundry Local provided a practical framework for this goal, motivating the design of a fully offline Retrieval-Augmented Generation system built from scratch. The diving theme was selected due to a personal interest in scuba diving; a one-star certification exam was completed successfully around the same period, making a diving-focused knowledge assistant a natural choice for the project's subject matter.

2. Key Concepts Learned

Retrieval-Augmented Generation (RAG)

A key insight gained through this project is that a language model does not need to "know" everything and it only needs the right information at the right moment. RAG retrieves relevant text from a document collection, adds it to the model's prompt as context, and lets the model generate an answer grounded in it. Through building DiveBuddy, it became clear how a small, local model can answer accurately about a narrow domain it was never trained on, and how it can be made to respond "I don't know" instead of guessing when the retrieved context is not relevant enough.

Embeddings and Vector Search

An embedding converts text into a vector of numbers that captures its meaning, so that similar texts produce similar vectors even without sharing the same words. The similarity between two vectors (measured via cosine similarity) determines which chunk is retrieved for a given question. This concept became concrete during testing, when a chunk mixing of several unrelated sentences was found to produce a diluted embedding, causing a relevant fact to be missed by retrieval. Splitting the paragraph into two resolved the issue, raising the similarity score from outside the top 5 results to rank 1.

SQLite for Local Storage

SQLite was selected as the storage solution because it is serverless — the entire database exists as a single file (rag.db), requiring no separate installation or configuration. It was used to store each document chunk alongside its embedding vector.

Prompt Engineering

It was observed that how a model is instructed matters almost as much as the information it is given. A single system prompt instruction to answer only from the given context and state uncertainty otherwise, was found to be the difference between a model that fabricates answers and one that remains reliable.

Foundry Local

This project provided first-hand experience running a language model entirely on a local machine rather than through a cloud API. Foundry Local's Python SDK closely resembles a hosted API in usage, which made the transition from cloud-based to on-device AI considerably more approachable than expected.

3. Development Process

A phased approach was followed in building DiveBuddy.

- **Setup and Foundations.** Foundry Local was installed and verified through a small Python script that loaded the qwen3-embedding-0.6b model and generated a test embedding. A basic project structure was then created, and cosine similarity was tested on a handful of example sentences to build an understanding of retrieval before implementing the full pipeline.
- **Data Ingestion.** An ingestion script (ingest.py) was developed to read .txt documents, split them into paragraph-level chunks, generate embeddings for each chunk, and store the results in a local SQLite database (rag.db). The knowledge base combines diving certification training notes with sections condensed from a 91-page Turkish Underwater Sports Federation guide diver handbook, translated into English.
- **Retrieval and Generation.** retrieval.py embeds an incoming question and ranks stored chunks by cosine similarity. app.py connects this to Foundry Local's phi-3.5-mini chat model: if the best-matching chunk scores above an empirically chosen similarity threshold (0.55), it is passed to the model as context along with a strict system prompt; otherwise, the assistant reports that it does not know rather than producing a fabricated answer.
- **Testing and Debugging.** The assistant was tested against direct questions, paraphrased questions, unanswerable questions, and edge cases such as empty input. This process surfaced two issues that's a chunk that diluted facts by mixing multiple topics, and short queries occasionally producing overly general answers the first of which was diagnosed and resolved.

4. Challenges and Solutions

Language Mismatch and Model Performance

The knowledge base was initially built in Turkish, in line with the source documents. However, testing revealed that the local chat model (phi-3.5-mini) produced noticeably lower-quality answers on Turkish text, and retrieval scores were also lower. The knowledge base, prompts, and test questions were subsequently translated into English, which measurably improved retrieval accuracy — the best-match cosine similarity on a comparable question rose from 0.733 to 0.831.

Chunk Granularity and Retrieval Accuracy

During testing, a question with a clearly correct answer in the documents ("Why do divers wear hoods?") was not retrieved within the top 5 results. Investigation revealed that the relevant sentence was embedded within a longer paragraph covering several unrelated topics, causing the embedding to represent an averaged, diluted meaning rather than the specific fact. Splitting the paragraph into two smaller, single-idea chunks resolved the issue, raising the relevant chunk's rank from outside the top 5 to rank 1 (similarity score 0.751). This was one of the more significant lessons of the project: how a knowledge base is chunked affects retrieval quality as much as the embedding model itself.

Similarity Threshold Calibration

A similarity threshold of 0.55 was chosen empirically after observing that answerable questions consistently scored above approximately 0.60, while unanswerable or unrelated questions scored between approximately 0.40 and 0.50. This threshold is what allows DiveBuddy to decline to answer rather than fabricate a response when relevant information is not present in the knowledge base.

Short or Ambiguous Queries

Single-word or vague queries (e.g., "diving") occasionally passed the similarity threshold while retrieving only a loosely related chunk, resulting in an overly general answer not fully grounded in the retrieved context. This was identified as a limitation of using only the single best-matching chunk (top_k=1) during generation, with increasing top_k alongside stricter prompt instructions identified as a possible future improvement.

5. Example Interactions

To demonstrate DiveBuddy's behavior in practice, three examples from the web interface are shown below.

Interface Overview

The DiveBuddy web interface, shown in its initial state before any question has been asked.

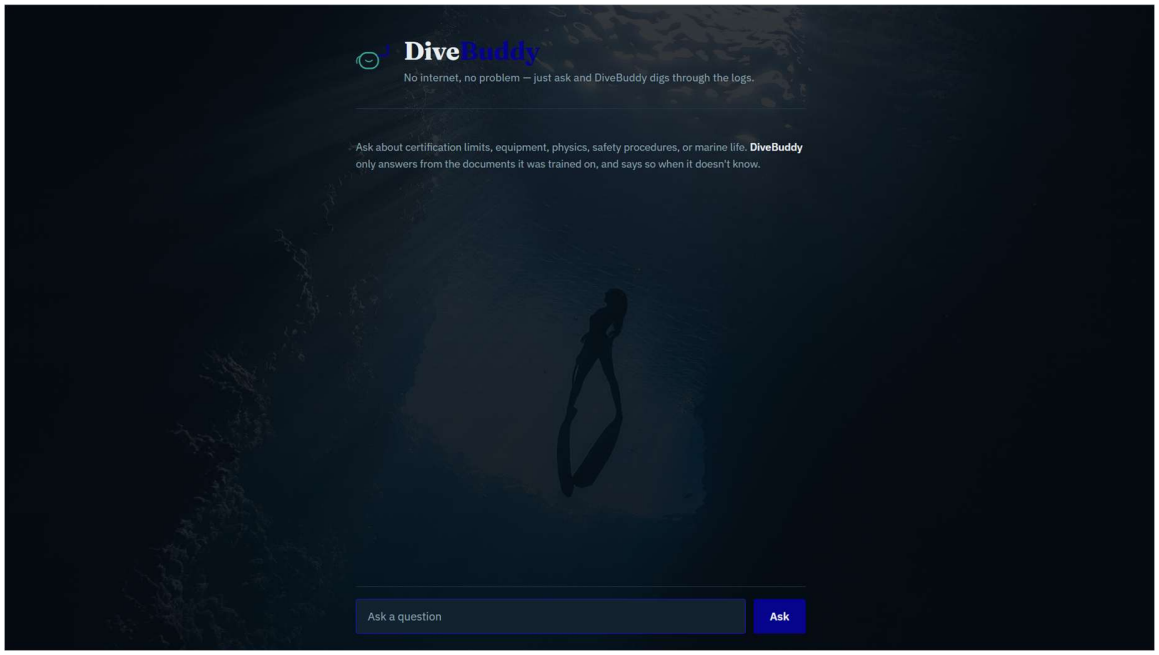

Figure 1: DiveBuddy Interface

Successful Retrieval

Two example questions with answers correctly grounded in the retrieved source document, with the source file displayed beneath each answer.

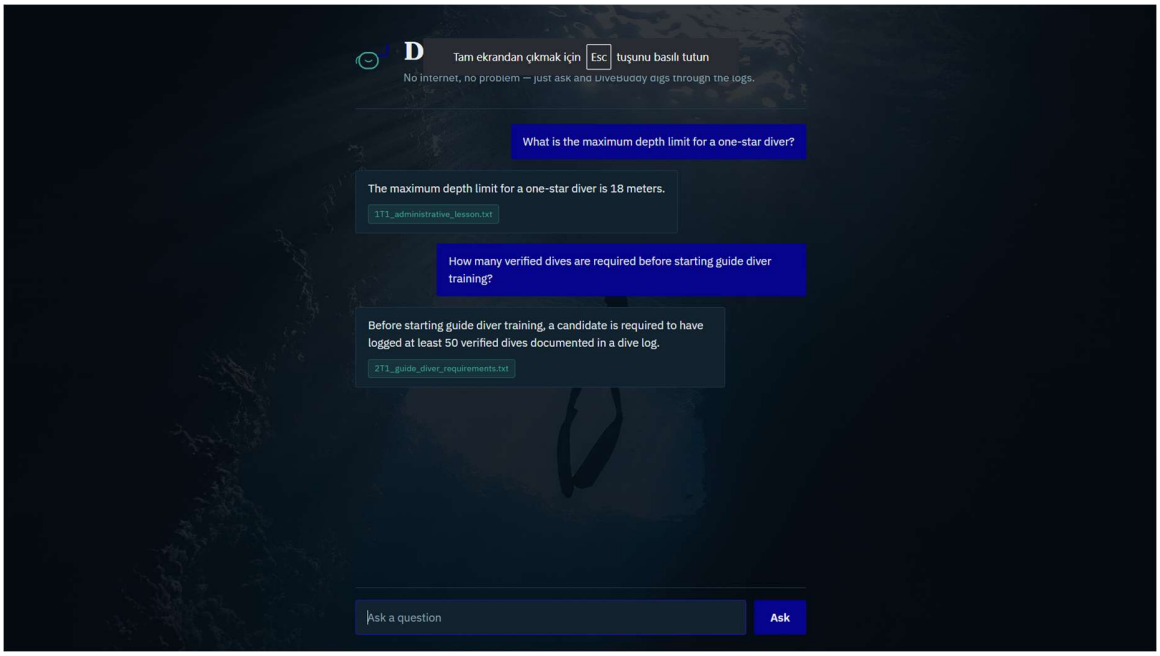

Figure 2: Correctly Answered Questions

Correct Refusal Behavior

Two example questions outside the scope of the knowledge base, both correctly declined rather than answered with fabricated information.

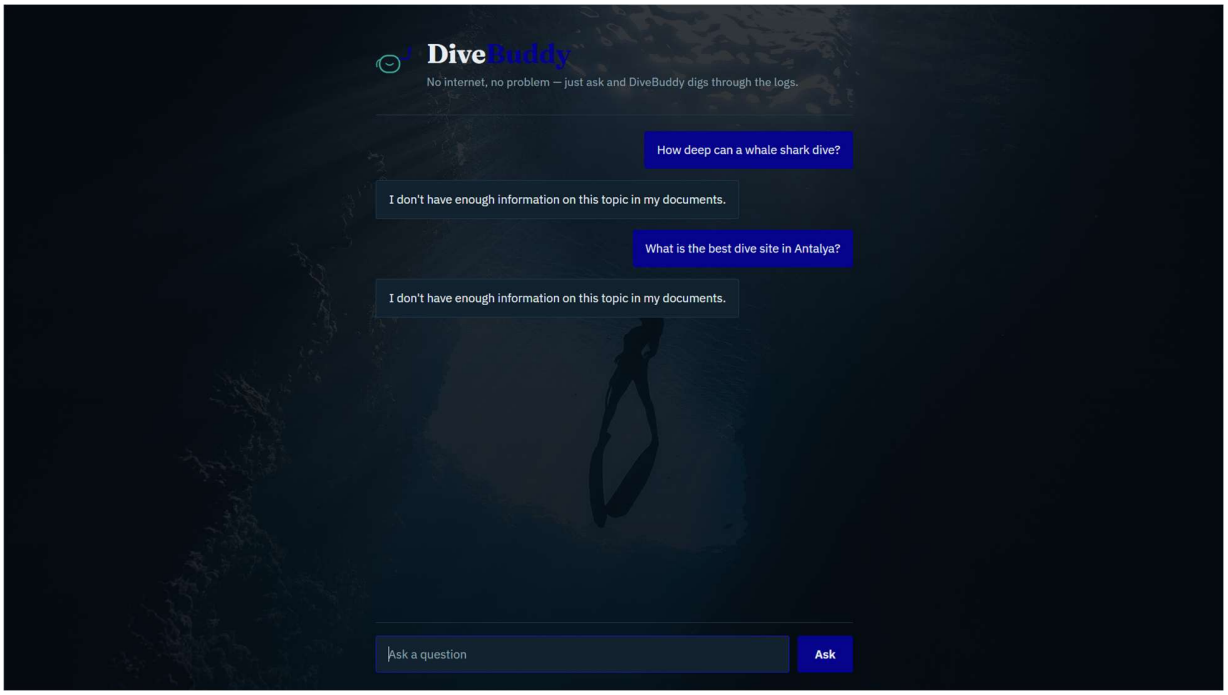

Figure 3: Questions Correctly Declined

6. Overall Evaluation

This project provided practical, end-to-end experience with building a Retrieval-Augmented Generation system using Microsoft's Foundry Local platform, from raw documents to a fully working, locally hosted assistant with both command-line and web interfaces. Manual testing across eleven cases covering direct questions, paraphrased questions, unanswerable questions, and edge cases showed that DiveBuddy correctly answered or correctly declined nine cases outright, with the two remaining cases investigated and, in one instance, resolved during testing.

Beyond the technical outcome, the project highlighted that the quality of a RAG system depends as much on data preparation decisions, including chunk size, chunking granularity, and language consistency, as on model selection itself. If repeated, chunk granularity would be audited systematically across the entire knowledge base from the start, rather than being discovered through targeted testing, and a larger top_k with more explicit prompt constraints would be used to better handle short or ambiguous queries.

More broadly, the program achieved its underlying goal: gaining hands-on, practical experience with Microsoft's AI and on-device inference tooling, moving from being a consumer of AI products such as Copilot to understanding how comparable systems are engineered.

7. References

- Building Your First Local RAG Application with Foundry Local (Microsoft Tech Community)
- What is Foundry Local? (Microsoft Learn)
- Tutorial: Build a RAG Application with Foundry Local (Microsoft Learn)
- SQLite Data Access (Microsoft Learn)
- Prompt Engineering Techniques (Microsoft Learn)
- TSSF (Turkish Underwater Sports Federation) Guide Diver Handbook

8. AI Usage Disclosure

AI assistance (Claude by Anthropic) was used throughout this project as a coding and project mentor, including explaining core concepts, debugging code, structuring the knowledge base, and developing the web interface. All testing, evaluation decisions, and conclusions presented are the author's own work.